OMID USA

AI FARMING DRONE

Dylan Serger, Eli Rindler, Amogh Silaparasetti, Ayushi Sharma, Huy Nguyen

Goals

Our goal is to help farmers in protecting their crops from diseases with the help of technology.

AI Team

We are developing an image/video processing AI to determine when and which crops need to be sprayed with the pesticide based on the conditions. The cameras will be attached to existing farming equipment and sends the data to cloud which will be run through our models

App Team

Previous UI was built to only support the test vineyard and the application did not work well on mobile

Create interactive UI to view sensor data and AI model response to crop images

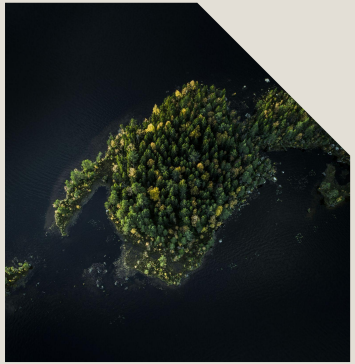
User Stories

Crop Health Monitoring

Farmers can use drones to detect defects and access crop health

Fertilizer Optimization

Ability to identify field areas needing extra fertilizer for precise application



Manual Control & Environment Data

Farmers can view live crop footage, manually spray pesticides when needed, and access environmental data like temperature and humidity

Intellectual Merits

Focused on innovation

1

Integration of AI

The project integrates AI-based crop analysis with drone systems for real-time monitoring on the crops

2

Fertilizer Optimization

Identifies specific field zones requiring care and support targeted fertilizer application.

3

Real-time Monitoring

Live drone sensor-application enables farmers to make intervention decisions

4

System Architecture

Scalable framework adaptable to different crop types and farm sizes

Broader Impacts

Increased Agricultural Efficiency

Lets farmers monitor large fields accurately.

Reduces time spent on manual field inspections.

Environmental Sustainability

Reduces excessive fertilizer use through precise application.

Minimizes pesticide over-spraying and lowering environmental contamination.

Scalability Applications

Adaptable to different crops.

Potential integration with smart farming ecosystem.

Design Diagrams

Diagram 0:

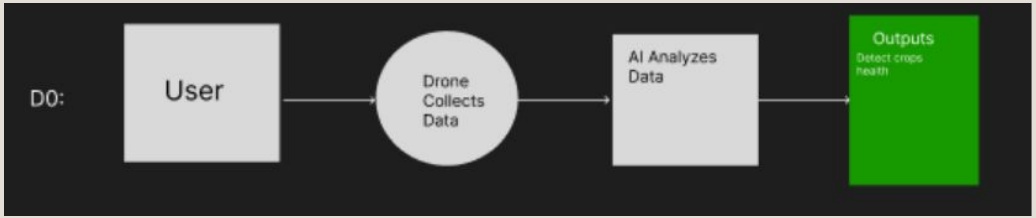
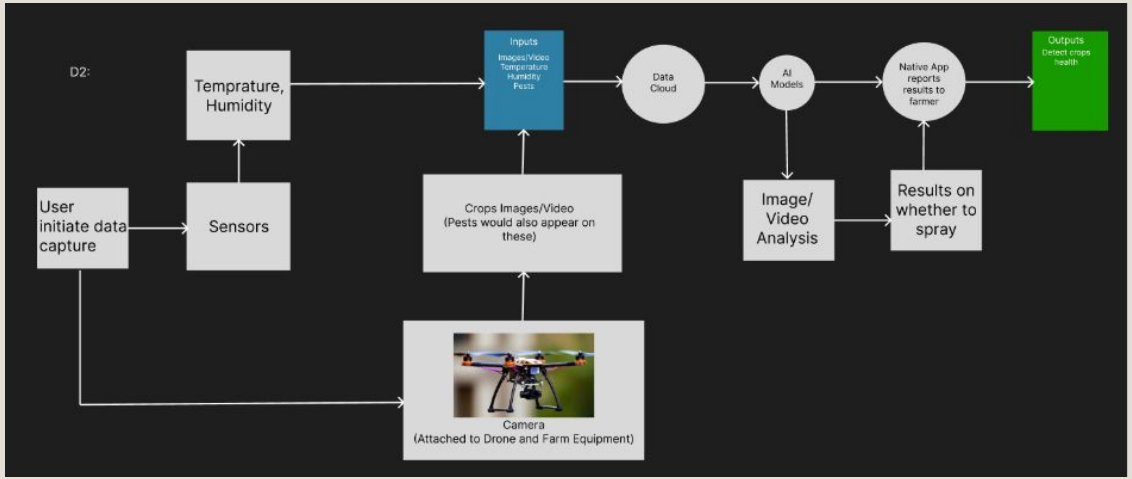
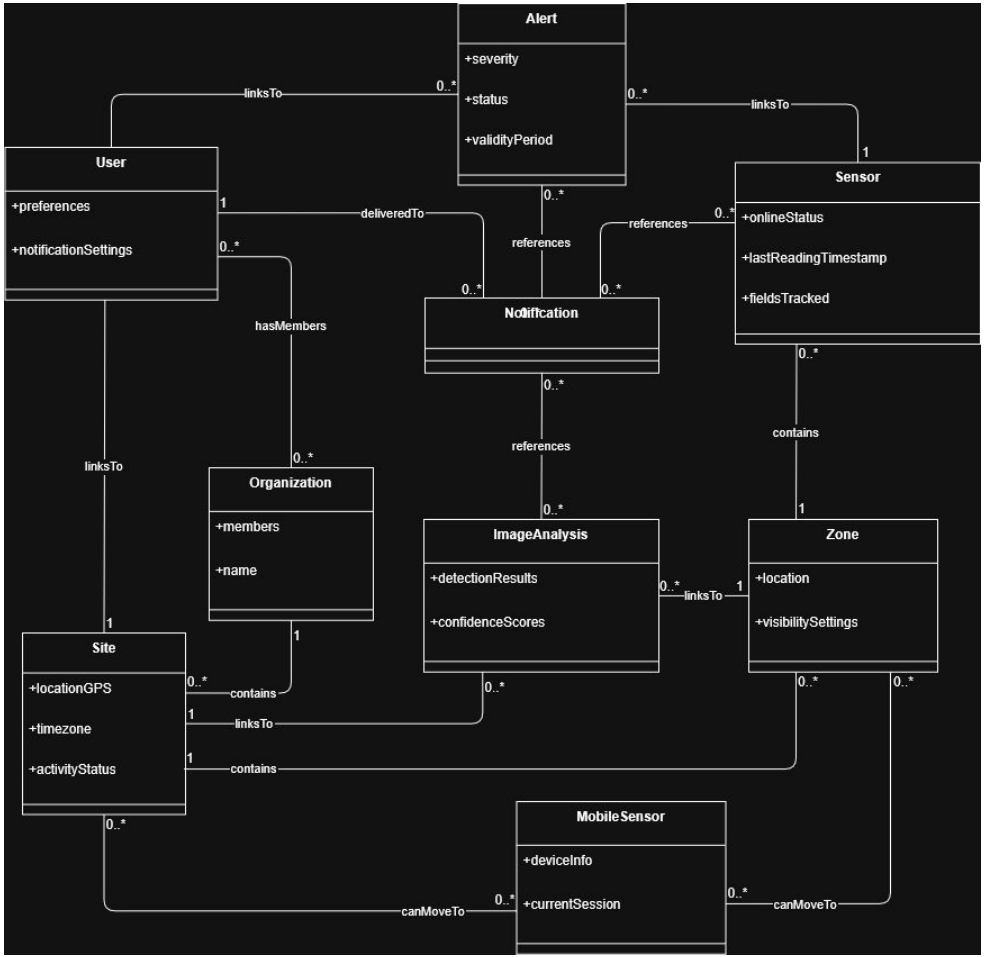


Diagram 1:

Diagram 2:



Data Model Diagram



Technology

Frontend:
- Flutter

Backend:
- Big Query
- FireBase
- Google Cloud Compute

For app deployment on both iOS & Android, we used Google Play Developer Console and iOS Developer Console.

For model inference, containerized pytorch packages run in Google Cloud Compute

Milestones

We're here!

Step 1

Initiate works. Upgrade Application UI

Step 2

Gather more data to train AI Models.

Step 3

Sensors connected with the app and give updates in real time.

Step 4

Deployment to Android and iOS and fix mobile sensors.

Goal

App working exceptionally on Mobile and Desktop platforms. Sensors able to detect crop diseases.

AI Pipeline

Flow of data and results for vine classification (Multispectral)

Capture

Multispectral Images captured

The images are taken via drone or by cameras mounted on tractors. Images are large 5 channel TIFF files

Prep

Transfer to cloud and align

TIFF images are processed in the cloud, and each 64 x 64 group of pixels is passed into the vine_presence model

Infer

For vine tiles, is this diseased?

The model will only look at tiles that are at least 50% likely to be grape vines. For those tiles, a percentage likelihood of disease is output

Report

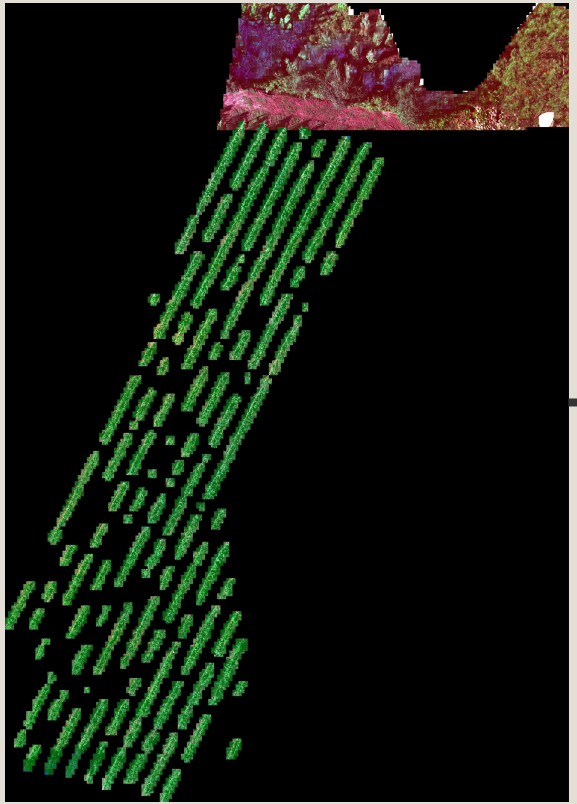
Heatmap published for farmer

The resulting prediction are stitched back over the original image and pushed into firestore. These images are used by the farmer to gauge crop health

AI Improvements

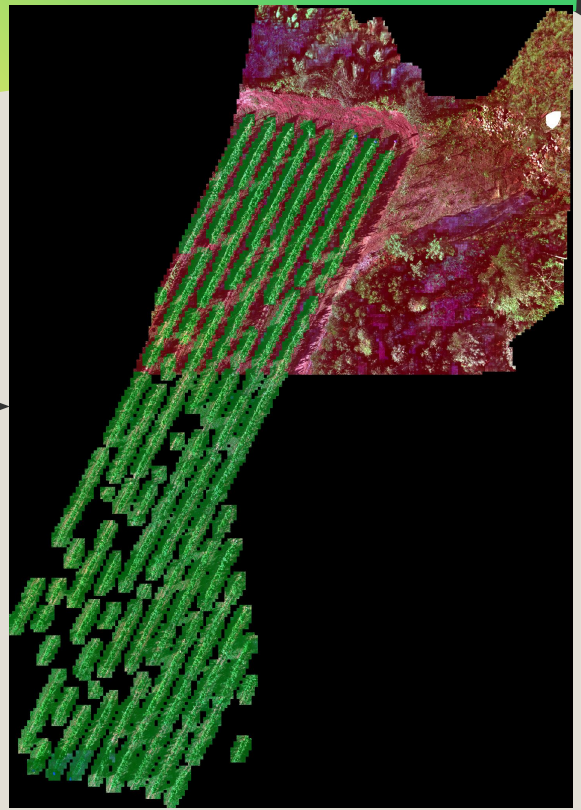
Changes to inherited architecture and workflow

Vine Presence Model
Improvements



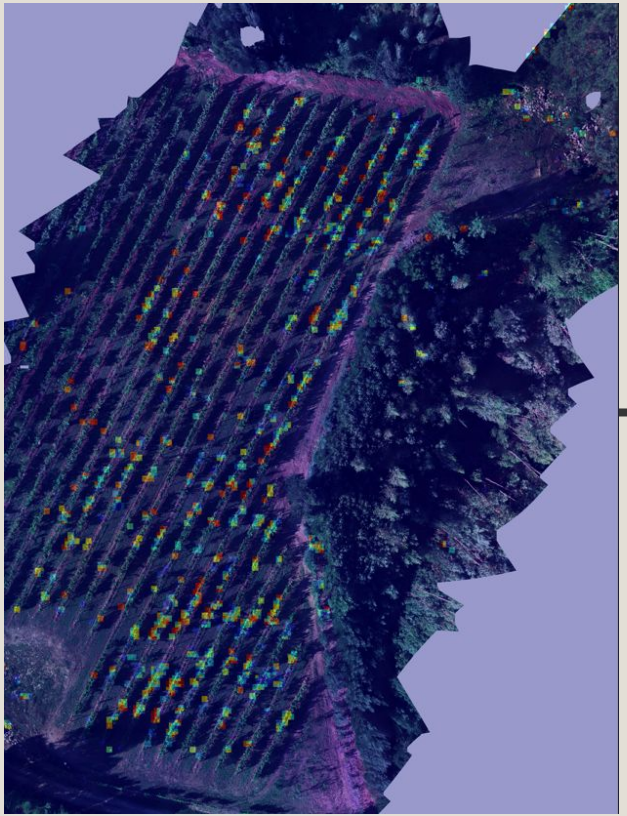
More Training Data

Better vine coverage



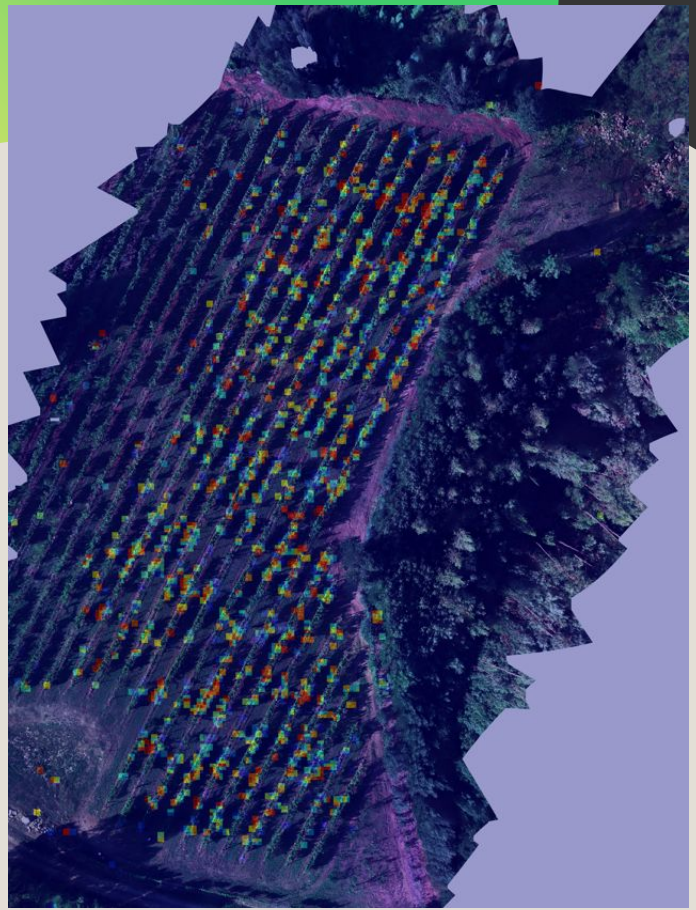
Vine presence training data prep

Vine Presence Model
Improvements



More Training Data

Better vine coverage
+
Better Precision
(Less FPs)



Inference output

Model Distillation

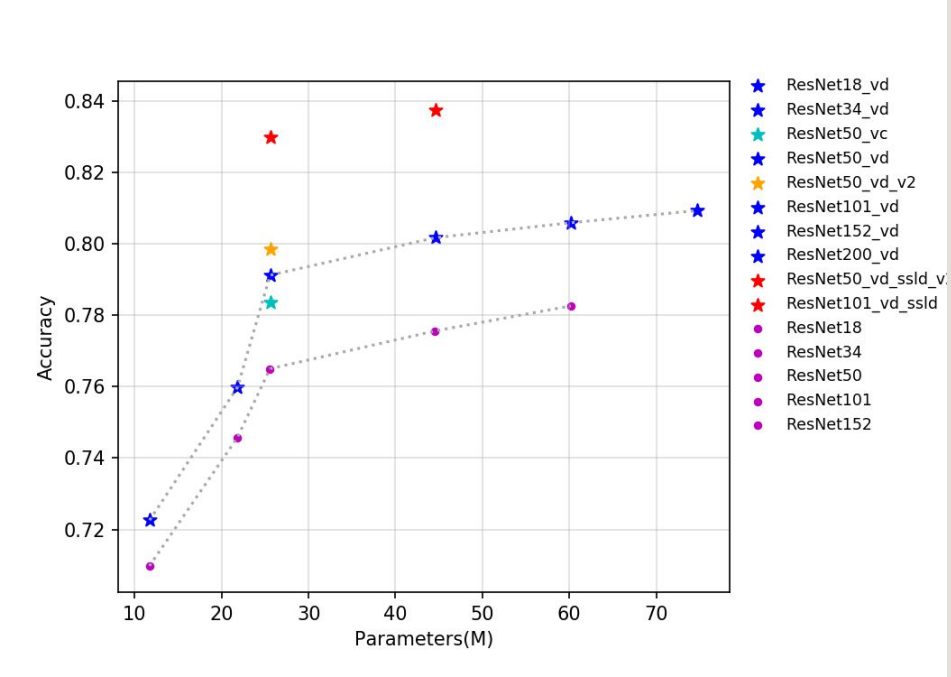
Original Approach

A simple ResNet18 for Disease classification. Though effective, will ultimately not have the capacity needed without enormous training sets

New Approach

Now we utilize ResNet50, ResNet101, Resnet152, and efficientnet_b4 (not shown on graph)

Together these learn an enormous amount about the training data (very much overfit). However, when used to teach a ResNet18 model, the distillation imparts a high level of accuracy with improved generalization.



Frost Pipeline

Flow of data and results for frost prediction
Just recently built and awaiting full deployment / testing

Capture

Every 15 minutes from every zone

Each zone (as many as the farmer needs) has a sensor box that delivers environment data to BigQuery.

Prep

Container grabs data + predictions

Upon insertion of the sensor data, a cloud container will fetch the latest data looking back 3 days, and any prediction it made 6 hours prior

Infer

Will it frost in the next 6 hours?

The model will only make a prediction on the following 6 hour window, and will utilize the result of the past 6 hours to perform back propagation against the previous prediction

Report

Predictions passed to WebApp

The resulting predictions for each zone are pushed back to BigQuery for history, and are shown in the web and mobile applications

Disease Identification

Flow of data and results for early-stage plant disease detection
Field testing in progress

Capture

Real-World Image Collection

Leaf images captured from everyday cameras in natural field conditions, focusing on early signs of disease.

Prep

Smart Image Processing

Isolate the leaf, enhance contrast, and highlight subtle spots, discoloration, or texture changes to make early disease patterns more visible.

Infer

Identify the disease

A Vision Transformer model analyzes the processed image to detect patterns and classify the type of disease with high accuracy.

Report

Actionable Results for Farmers

Predictions with confidence scores are delivered, allowing farmers to take early action. The system can continuously improve as more field data is collected.

Disease Detection Model
Preprocessing

Highlight Disease Markers

Reduce Background Noise

Leaf preprocessing example

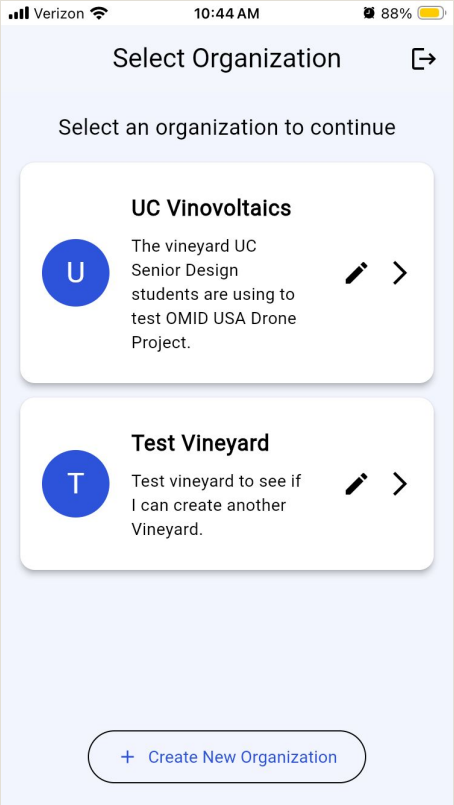
Use ViTi to identify disease

Prediction:
Black Rot — 92% confidence

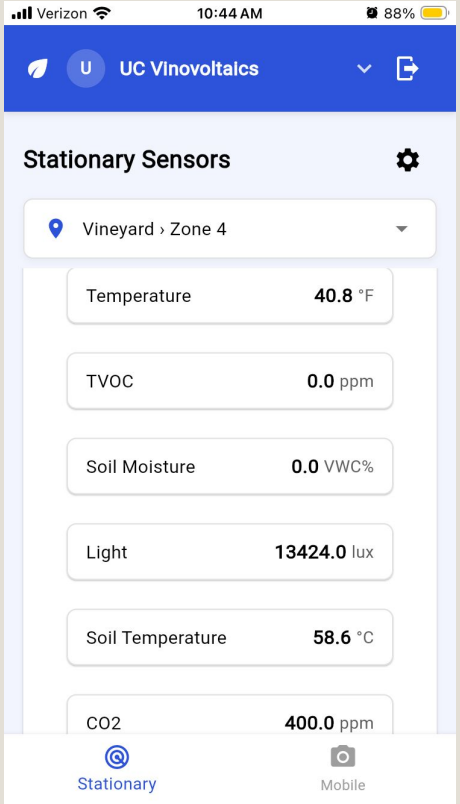
Disease identification example

CURRENT APP

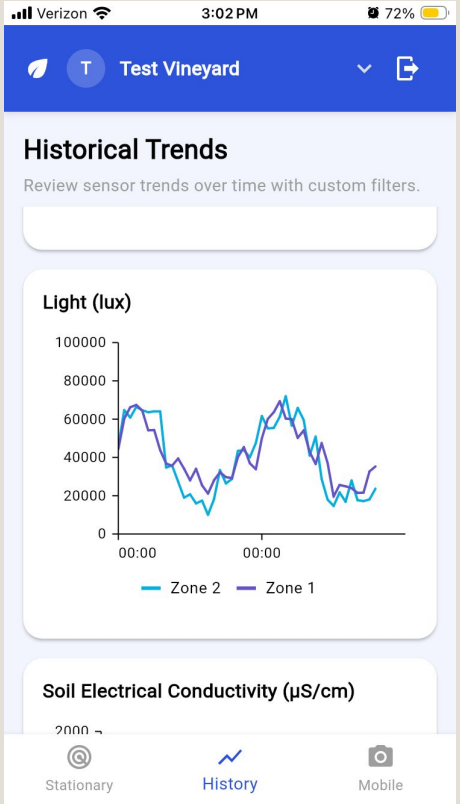
Login | Organization Selection



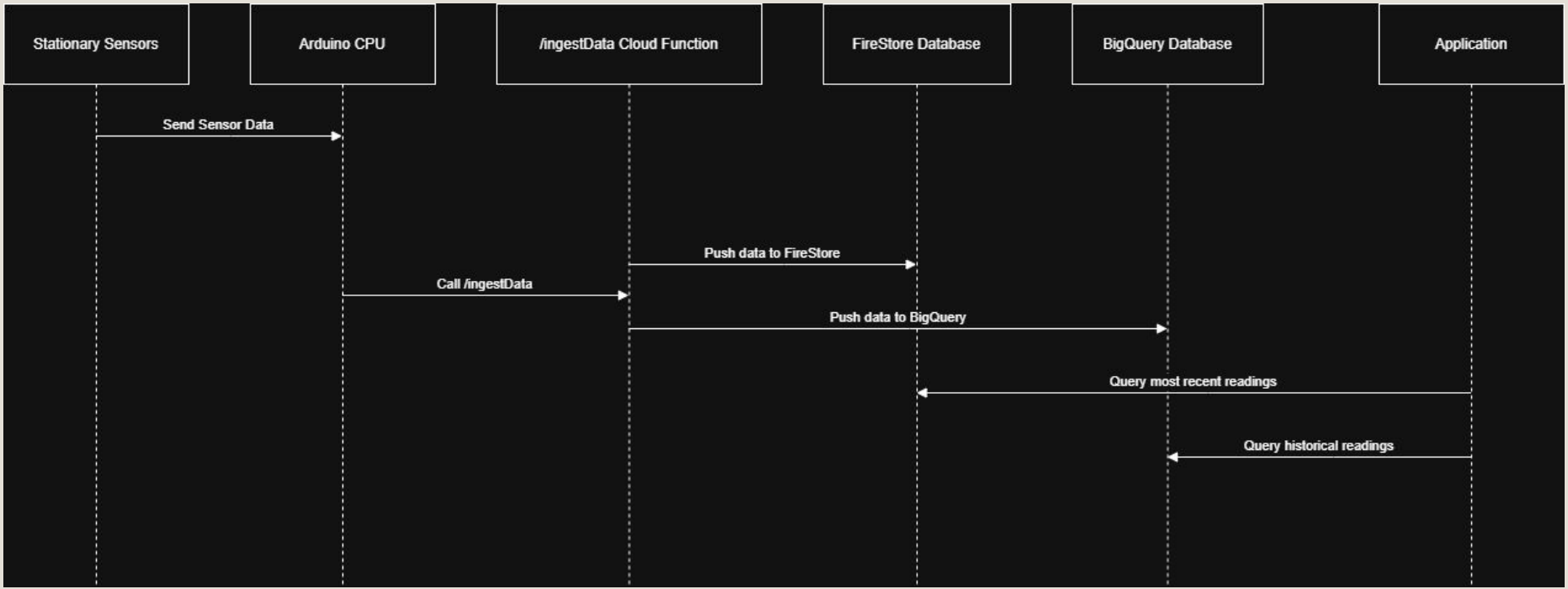
Stationary Sensors Current Readings



Stationary Sensors Historical Readings



Data Flow


Stationary Sensors
Arduino CPU
/ingestData Cloud Function
FireStore Database
BigQuery Database
Application
Send Sensor Data
Push data to FireStore
Call /ingestData
Push data to BigQuery
Query most recent readings
Query historical readings

Challenges

Some challenges we faced

1

Limited Dataset of Images of Pests/Diseases the crops could be infected with.

2

Retrieving Data from Sensors such as temperature, humidity, soil moisture, etc.

3

Try to combine the AI Models that are currently being developed for various purposes and ensure that the input and output are of same type.

Next Steps

Ready to pick-up

1

Mobile

Update Mobile Sensor (Drone) Page of App

2

Models

Add ML Models Integration to App

3

Images

Add Ability for Farmer to Push Images to the App

Thank you!
Any questions?